

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

CURSO: INGENIERÍA DE SOFTWARE I (IF614)

Centro de Capacitación e Inducción CGB Academy

Informe de Ingeniería de Software: Requisitos, Arquitectura y Método Híbrido

Entidad Beneficiaria:

CGB Academy (CIIP LATAM, GEOMINA LATAM y BIOMEDIC)

Integrantes del Equipo de Desarrollo:

Barazorda Cuellar, Hector	<i>Scrum Master / Dev Full-Stack</i>
Carpio Hermoza, Alex	<i>Desarrollador Full-Stack</i>
Quispe Mamani, Domingo de Guzman	<i>Desarrollador Full-Stack</i>
Saire Bustamante, Edmil Jampier	<i>Product Owner / Dev Full-Stack</i>
Ticona Jancco, Ronaldo	<i>Desarrollador Full-Stack</i>

Índice

1. Datos Generales del Proyecto	3
2. Objetivo General y Específicos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Alcance del Sistema	4
3.1. Funcionalidades Incluidas (In-Scope – MVP)	4
3.2. Exclusiones Expresas (Out-of-Scope)	4
4. Justificación del Proyecto	4
4.1. Justificación Práctica y Organizacional	4
4.2. Justificación Académica y de Ingeniería	4
5. Contexto del Proyecto y Restricciones	4
5.1. Contexto Operativo	4
5.2. Restricciones del Sistema (Mapeo Taxonómico ISO/IEC 25010:2023)	5
6. Tecnologías Utilizadas y Fundamentación Arquitectónica	5
7. Stakeholders (Matriz de Interesados)	7
8. Enfoque de Desarrollo Seleccionado y Justificación	7
9. Prácticas Adoptadas del Proceso	7
9.1. Aportes de Scrum	7
9.2. Aportes de Kanban y Límites WIP	8
9.3. Aportes de Extreme Programming (XP) y Ciclo TDD	8
9.4. Uso de Inteligencia Artificial en el Proceso	8
10. Roles, Responsabilidades y Forma de Coordinación	8
10.1. Política de Roles Rotativos por Sprint	8
10.2. Estrategia de Ramas en Git y Flujo para Desarrolladores	9
11. Flujo de Trabajo del Equipo	9
11.1. Estructura del Tablero Kanban Oficial y Límites WIP	9
11.2. Justificación Matemática de los Límites WIP (Ley de Little)	10
11.3. Unidad de Flujo y Subtareas Técnicas TDD	10
11.4. Políticas Explícitas de Transición entre Columnas	11
11.5. Protocolo de Gestión de Bloqueos e Impedimentos	11
11.6. Dinámica por Sprints y Evolución del Tablero	11
11.6.1. Sprint 1 (HU-001 a HU-006): Núcleo de Capacitación y Acceso Público	11
11.6.2. Sprint 2 (HU-007 a HU-012): Estructura Organizacional y Onboarding	12
11.6.3. Sprint 3 (HU-013 a HU-015): Microtests Formativos y Supervisión . .	12

11.7. Política WIP Oficial del Equipo	12
11.8. Flujo Metodológico Integrado y Definition of Done	13
12.Incrementos Presentados del Proyecto	13
12.1. Incremento 1 (Sprint 1: Núcleo de Capacitación y Acceso Público)	13
13.Métricas y Seguimiento del Proceso	13
14.Acuerdos de Mejora para el Siguiente Incremento	14
15.Política de Uso de IA y Registro de Uso	14
16.Conclusiones	14

1. Datos Generales del Proyecto

- **Denominación del Sistema:** Centro de Capacitación e Inducción CGB Academy.
- **Organización Beneficiaria:** CGB Academy, integrando los programas de CIIP LATAM, GEOMINA LATAM y BIOMEDIC sin generar plataformas dispersas.
- **Repositorio Central en GitHub:** <https://github.com/milith0kun/proyecto-ing-software>
- **Ramas de Trabajo:** `main` (código estable de producción) y `develop` (integración continua colaborativa).
- **Alcance del MVP:** 15 Historias de Usuario funcionales agrupadas en 3 Sprints de dos semanas de duración cada uno.

2. Objetivo General y Específicos

2.1. Objetivo General

Diseñar, implementar y validar el **Centro de Capacitación e Inducción CGB Academy**, una solución web modular orientada a la capacitación de colaboradores internos y a la orientación pública de estudiantes y docentes, estructurada en diapositivas interactivas (*slides*), rutas de aprendizaje organizacional jerárquicas y microtests formativos, prescindiendo de la complejidad operativa de un LMS tradicional.

2.2. Objetivos Específicos

1. Especificar y priorizar 15 historias de usuario funcionales para el MVP conforme al estándar internacional **ISO/IEC/IEEE 29148:2018** [8] y el marco INVEST [3], con criterios de aceptación verificables en formato BDD (*Dado que / Cuando / Entonces*).
2. Diseñar una arquitectura web ligera, desacoplada y responsiva conforme a las áreas de conocimiento de Arquitectura y Construcción de Software de **SWEBOK V4.0** [6], apta para desplegarse sobre una infraestructura VPS objetivo de 2 vCPU y 8 GB de RAM.
3. Implementar el motor de creación y publicación de capacitaciones mediante *slides* administrables sin requerir conocimientos de programación (*no-code*).
4. Modelar el módulo organizacional de onboarding relacionando $\text{Área} \rightarrow \text{Puesto} \rightarrow \text{Ruta} \rightarrow \text{Capacitación}$, garantizando activación segura y registro automático de progreso.
5. Incorporar microtests formativos con retroalimentación inmediata orientados al aprendizaje sin penalizaciones punitivas.
6. Aplicar un método ágil híbrido combinando la cadencia de Scrum [14], el control de flujo Kanban con límites WIP fundamentados en la Ley de Little [11, 1] y las disciplinas de ingeniería de XP/TDD [2].

3. Alcance del Sistema

3.1. Funcionalidades Incluidas (In-Scope – MVP)

- **Sprint 1 (Motor y Acceso Público):** Autenticación con control de acceso por roles (Admin/Colaborador), creador de capacitaciones, editor interactivo de slides (texto, imágenes, listas, indicaciones, botones), previsualización, gestión de estados (Borrador/Publicada) y catálogo público libre sin autenticación.
- **Sprint 2 (Onboarding Interno):** Mantenimiento de áreas y puestos con integridad referencial, construcción de rutas de onboarding, registro de colaboradores, activación mediante token seguro de uso único y módulo privado “Mi Onboarding”.
- **Sprint 3 (Microtests y Supervisión):** Creación de evaluaciones cortas de opción múltiple, resolución con retroalimentación inmediata al colaborador y panel administrativo para supervisar el avance individual y colectivo.

3.2. Exclusiones Expresas (Out-of-Scope)

Se excluyen deliberadamente para el MVP: venta de cursos, carritos de compra y pasarelas de pago; aulas virtuales en vivo; emisión automatizada de diplomas o certificados oficiales; almacenamiento de video pesado o biblioteca de descargas masivas; soporte multi-idioma (versión 1.0 exclusivamente en español); y migración de historiales legados.

4. Justificación del Proyecto

4.1. Justificación Práctica y Organizacional

CGB Academy agrupa la formación especializada de CIIP LATAM, GEOMINA LATAM y BIOMEDIC. Previamente, la inducción de personal dependía de documentos manuales en PDF y explicaciones repetitivas no trazables. La plataforma automatiza la inducción, estandariza procesos corporativos y ofrece a la comunidad externa un centro de consulta claro y accesible en todo momento.

4.2. Justificación Académica y de Ingeniería

El proyecto aplica de manera rigurosa los estándares de la ingeniería de software moderna [15, 13, 6]: elicitación formal de requisitos mediante historias de usuario atómicas bajo ISO/IEC/IEEE 29148 [8], diseño de arquitecturas eficientes para recursos limitados, control de versiones bajo GitFlow y aseguramiento continuo de la calidad asistido por TDD.

5. Contexto del Proyecto y Restricciones

5.1. Contexto Operativo

El sistema prestará servicio a nivel nacional e internacional vía web. Los colaboradores realizarán su onboarding en horarios laborales desde sus computadoras de trabajo o dispositivos

móviles, mientras que el público externo consultará libremente contenidos informativos sin fricción de registro.

5.2. Restricciones del Sistema (Mapeo Taxonómico ISO/IEC 25010:2023)

Los diez Requerimientos No Funcionales (RNF) del sistema se clasifican rigurosamente conforme al modelo de calidad de producto de la norma internacional **ISO/IEC 25010:2023** (Serie **SQuaRE**) [7]:

Código	Requerimiento No Funcional	Característica ISO 25010	Métrica de Aceptación / Estándar
RNF-001	Diseño Responsive	Usabilidad / Interacción	Adaptación fluida ($\geq 360\text{px}$) sin pérdida funcional.
RNF-002	Idioma de Operación	Usabilidad / Comprensibilidad	Interfaz 100 % en español neutral sin mezclas lingüísticas.
RNF-003	Infraestructura Objetivo	Eficiencia de Rendimiento	Despliegue en VPS (2 vCPU / 8 GB RAM) sin degradación de memoria.
RNF-004	Contenido Ligero	Eficiencia de Rendimiento	Carga $< 1,5$ s en 4G; compresión en WebP/SVG y sin videos pesados.
RNF-005	Seguridad de Acceso Interno	Seguridad (Confidencialidad)	Rutas privadas protegidas por sesión; denegación con HTTP 401/403.
RNF-006	Protección de Credenciales	Seguridad (NIST SP 800-218 [16])	Hashing bcrypt (costo ≥ 10) y tokens de uso único con caducidad.
RNF-007	Minimización de Datos	Privacidad por Diseño	Captura exclusiva de datos indispensables para la inducción corporativa.
RNF-008	Consistencia Visual	Mantenibilidad / Modularidad	Uso del sistema de componentes estandarizado de CGB Academy.
RNF-009	Facilidad de Administración	Usabilidad / Operabilidad	Gestión intuitiva de slides <i>no-code</i> sin edición manual de código.
RNF-010	Integridad de Información	Fiabilidad / Consistencia	Restricciones contra borrado huérfano de áreas/puestos con personal activo.

Tabla 1: Mapeo formal de restricciones del sistema según la norma ISO/IEC 25010:2023.

6. Tecnologías Utilizadas y Fundamentación Arquitectónica

Bajo el marco del **SWEBOK V4.0** [6], la arquitectura se diseñó aplicando patrones reconocidos de ingeniería de software:

Capa / Componente	Tecnología	Justificación Técnica y Patrones de Ingeniería
Base de Datos	MongoDB Atlas	NoSQL documental en la nube (Cluster0), óptima para estructuras polimórficas de slides y trazas de onboarding.
ORM / Acceso a Datos	Prisma ORM	Modelado tipado con conector MongoDB. Implementa el patrón Singleton [5] en <code>src/lib/prisma.ts</code> para evitar el agotamiento de sockets en el pool.
Arquitectura Full-Stack	Next.js 16 (App Router)	Arquitectura limpia unificada: Backend (Route Handlers en <code>/app/api/...</code>) y Frontend (React Server Components y Client Components) con motor Turbo-pack .
Aplicación Web	Next.js Web App	Optimización para SEO y rendimiento (<i>Core Web Vitals</i>) con carga instantánea en navegación pública y privada.
Estilos UI	TailwindCSS / CSS Moderno	Sistema de diseño basado en utilidades que garantiza consistencia visual y supresión de estilos no utilizados (<i>purging</i>).
Lenguaje	TypeScript	Tipado estático integral que previene defectos en tiempo de compilación entre modelos, endpoints y vistas.
Seguridad de Entorno	Variables <code>.env</code>	Aislamiento de credenciales y secretos fuera del repositorio Git, cumpliendo con el estándar NIST SSDF SP 800-218 [16].
Control de Versiones	Git & GitHub	Ramas: principal (<code>edmil-saire</code>), desarrollo (<code>desarrollo</code>) e individuales por desarrollador.
Gestión de Proyecto	Jira / GitHub Projects	Tablero Kanban con límites WIP fundamentados, Planning Poker [3] y métricas de flujo.

Tabla 2: Pila tecnológica seleccionada con fundamentación arquitectónica SWEBOK v4.0.

7. Stakeholders (Matriz de Interesados)

Actor / Interesado	Tipo	Influencia	Expectativa Principal
Colaborador Interno	Directo	Alta	Experiencia de onboarding clara, interactiva y sin fricciones técnicas.
Administrador	Directo	Alta	Creación sencilla de slides, asignación de rutas y visualización de progreso.
Público General	Externo	Media	Consulta rápida y abierta de contenidos de inducción sin requerir registro.
Directorio CGB	Clave	Crítica	Reducción de costos de inducción, cumplimiento del cronograma y estabilidad.
Equipo de Desarrollo	Interno	Alta	Código limpio, arquitectura mantenible y entregas incrementales verificadas.
Docente Evaluador	Académico	Crítica	Rigor metodológico en ingeniería de software y software completamente operativo.

Tabla 3: Matriz de stakeholders del proyecto.

8. Enfoque de Desarrollo Seleccionado y Justificación

Se ha adoptado un **Método Ágil Híbrido** [15, 13]:

- **Descarte de Cascada:** Se descartó por su inviabilidad ante ajustes dinámicos de interfaz y requerimientos en sprints de corta duración.
- **Estructura Base Scrum** [14]: Provee la cadencia de trabajo en sprints de 2 semanas, ceremonias de planificación, Daily, Review y Retrospectiva.
- **Control de Flujo Kanban** [1, 10]: Aporta visibilidad en tiempo real mediante un tablero de cuatro columnas, límites de trabajo en progreso (WIP) justificados en la teoría de colas y gestión explícita de bloqueos.
- **Prácticas de XP y TDD** [2, 12, 4]: Garantiza la calidad técnica mediante desarrollo guiado por pruebas, integración continua y refactorización orientada a la contención de la deuda técnica.

9. Prácticas Adoptadas del Proceso

9.1. Aportes de Scrum

Sprints de 2 semanas con cadencia regular para completar las 15 historias de usuario del MVP. Ceremonias formales obligatorias: Sprint Planning con estimación mediante Planning Poker [3], Daily Scrum de 15 minutos para sincronización diaria, Sprint Review con demostración en vivo y Sprint Retrospective para la formulación de compromisos de mejora continua y acuerdos de rotación de roles.

9.2. Aportes de Kanban y Límites WIP

- **Tablero Visual de Cuatro Columnas:** Estructura controlada: TO DO → DESARROLLO [WIP 3] → REVISIÓN [WIP 2] → DONE.
- **Límites WIP Justificados:** Para los 3 desarrolladores activos, se establece WIP = 3 en Desarrollo (1 desarrollador = máximo 1 HU activa) y WIP = 2 en Revisión.
- **Principio de Fluidez:** Al alcanzarse la capacidad de Revisión (2/2), la prioridad del equipo cambia inmediatamente: suspender el inicio de nuevo trabajo y colaborar para finalizar y fusionar las historias en revisión (*“Stop starting, start finishing”*).
- **Visibilidad de Impedimentos:** Bloqueos gestionados mediante etiqueta BLOQUEADA en la columna actual, sin crear columnas redundantes.

9.3. Aportes de Extreme Programming (XP) y Ciclo TDD

Cada Historia de Usuario desglosa internamente cuatro subtarefas técnicas bajo el ciclo de Desarrollo Guiado por Pruebas:

1. **T1 – Pruebas / RED:** Escritura inicial de pruebas unitarias o de integración que fallan.
2. **T2 – Implementación / GREEN:** Codificación mínima necesaria para superar las pruebas.
3. **T3 – Refactorización:** Optimización de estructura y legibilidad sin alterar el comportamiento.
4. **T4 – Validación e Integración:** Verificación de criterios BDD (*Dado/Cuando/Entonces*) y preparación del Pull Request.

Complementado con Integración Continua (CI), revisión obligatoria por pares (*Peer Review*) y diseño simple.

9.4. Uso de Inteligencia Artificial en el Proceso

Uso de modelos de lenguaje para asistencia en generación de fixtures, casos de prueba y esquemas Prisma, bajo el principio de **validación humana obligatoria**: ningún miembro incorpora código sin comprenderlo y probarlo rigurosamente.

10. Roles, Responsabilidades y Forma de Coordinación

10.1. Política de Roles Rotativos por Sprint

El equipo adopta una estructura de **roles rotativos por iteración** para fomentar el aprendizaje integral de todos los integrantes:

- **Sprint 1 (Sprint Piloto de Evaluación):**

- **Product Owner:** Saire Bustamante, Edmil Jampier. Responsable de la priorización del backlog, validación de criterios BDD (*Dado/Cuando/Entonces*) y desarrollo full-stack.
 - **Scrum Master:** Barazorda Cuellar, Hector. Responsable de facilitar ceremonias (Daily de 15 min), remoción de bloqueos técnicos y desarrollo full-stack.
 - **Desarrolladores Full-Stack:** Carpio Hermoza, Alex; Quispe Mamani, Domingo de Guzman; Ticona Jancco, Ronaldo. Ejecución autónoma de sus HUs asignadas y revisión por pares.
- **Evaluación y Rotación en Retrospectiva:** Al cierre del Sprint 1, el equipo analiza la dinámica y efectividad de las acciones para determinar la rotación de los roles de Product Owner y Scrum Master hacia los demás integrantes en los Sprints 2 y 3.

10.2. Estrategia de Ramas en Git y Flujo para Desarrolladores

Para garantizar orden en el código y evitar colisiones, se define el siguiente esquema de ramas:

- **Rama Principal (edmil-saire):** Rama *main* del repositorio oficial, destinada exclusivamente a versiones estables y libres de errores.
- **Rama de Integración (desarrollo):** Espacio común donde convergen los avances para testear la unión de los desarrolladores antes de producción.
- **Ramas Individuales por Desarrollador:** Cada miembro dispone de una rama con su nombre (*hector-barazorda*, *alex-carpio*, *domingo-quispe*, *edmil-saire*, *ronaldo-ticona*).
- **Flujo Operativo:** Cada desarrollador ejecuta `git pull origin desarrollo` en su rama para sincronizar cambios, sube commits con su cuenta de GitHub a su rama personal y genera un Pull Request hacia *desarrollo*. Tras el testeo conjunto, se integra a *edmil-saire*.

11. Flujo de Trabajo del Equipo

11.1. Estructura del Tablero Kanban Oficial y Límites WIP

El flujo se organiza formalmente en cuatro estados con límites explícitos de trabajo en progreso:

Columna	Significado Operativo	Límite WIP
TO DO	Historias del Sprint comprometidas aún no iniciadas.	Sin límite
DESARROLLO	Historias en codificación activa y construcción de pruebas.	3
REVISIÓN	Desarrollo culminado; en espera de revisión por pares y testing de integración.	2
DONE	Historias integradas en <i>desarrollo</i> y aceptadas por el Product Owner.	Sin límite

Tabla 4: Estructura de columnas y políticas WIP del tablero Kanban.

11.2. Justificación Matemática de los Límites WIP (Ley de Little)

Desde la teoría matemática de colas, el comportamiento del flujo de trabajo se fundamenta en la **Ley de Little** [11, 1]:

$$WIP = TH \times CT \implies CT = \frac{WIP}{TH} \quad (1)$$

Donde TH representa el *Throughput* (velocidad de entrega de historias terminadas por unidad de tiempo) del equipo de 3 desarrolladores activos (TH cuasi-constante) y CT el *Cycle Time* promedio por historia. Matemáticamente, cualquier aumento descontrolado del WIP dilata en proporción directa el tiempo de ciclo (*Cycle Time*), generando esperas y multitarea ineficiente.

- **Desarrollo = 3:** Dado que el equipo cuenta con 3 integrantes en el rol Development, la regla es estricta: **1 desarrollador = máximo 1 HU activa en Desarrollo**. Previene la sobreproducción (*Muda* en Lean) y focaliza el esfuerzo en entregas verticales terminadas.
- **Revisión = 2:** Actúa como *buffer* óptimo de aseguramiento de la calidad. Si la columna se satura (2/2), la prioridad del equipo cambia inmediatamente: suspender el inicio de nuevo trabajo y abocarse colaborativamente a revisar y fusionar las historias pendientes (*“Stop starting, start finishing”*).

11.3. Unidad de Flujo y Subtareas Técnicas TDD

La tarjeta principal que transita por el tablero visual es la **Historia de Usuario** (por ejemplo, HU-002: **Crear una capacitación**) conforme al estándar **ISO/IEC/IEEE 29148:2018** [8]. No se crean tarjetas separadas en el tablero para cada tarea técnica.

Para materializar las disciplinas de la Programación Extrema (XP) postuladas por Beck [2] y los estándares de aseguramiento de calidad de ISTQB [9], cada Historia de Usuario en Jira desglosa internamente cuatro subtareas técnicas obligatorias:

Subtarea Técnica	Actividad en el Ciclo TDD	Fundamento de Calidad
T1 – Pruebas / RED	Diseñar y programar pruebas que inicialmente fallan por ausencia de código.	Prevención de defectos por diseño (<i>Defect Prevention</i> - ISTQB).
T2 – Implementación / GREEN	Programar la lógica mínima necesaria para hacer pasar las pruebas con éxito.	Diseño simple y eliminación de sobre-ingeniería (principio YAGNI).
T3 – Refactorización	Reestructurar el código interno sin alterar su comportamiento funcional.	Contención proactiva de la deuda técnica [12, 4].
T4 – Validación e Integración	Ejecutar pruebas completas, validar criterios BDD y preparar el Pull Request.	Integración continua y verificación formal de la Definition of Done.

Tabla 5: Subtareas técnicas obligatorias bajo el ciclo TDD integradas en cada Historia de Usuario.

11.4. Políticas Explícitas de Transición entre Columnas

Para que una tarjeta pueda avanzar entre columnas, debe cumplir estrictamente las siguientes políticas:

- **TO DO** → **DESARROLLO**: Pertenece al Sprint actual aprobado; suficientemente refinada y estimada; cuenta con criterios de aceptación verificables en formato BDD; el desarrollador comprende el alcance técnico; y **existe capacidad disponible dentro del límite WIP de Desarrollo** (≤ 3).
- **DESARROLLO** → **REVISIÓN**: Las pruebas automáticas fueron escritas y superadas ($T1$ y $T2$); se ejecutó la refactorización ($T3$); las pruebas locales pasan al 100 %; el código está subido a la rama personal de GitHub; y **existe capacidad disponible en el límite WIP de Revisión** (≤ 2).
- **REVISIÓN** → **DONE**: El código ha sido revisado y aprobado formalmente por pares (*Peer Review*); las pruebas de integración en **desarrollo** pasan sin regresiones; no existen errores bloqueantes de compilación; se validaron los criterios de aceptación con el Product Owner; y el Pull Request ha sido fusionado limpiamente.

11.5. Protocolo de Gestión de Bloqueos e Impedimentos

Para preservar la integridad del flujo y no distorsionar las métricas de tiempo de ciclo (*Cycle Time*), el equipo **no utiliza una columna separada llamada “Bloqueado”**. Cuando una historia de usuario queda impedida:

1. La tarjeta permanece en su columna física actual (Desarrollo o Revisión).
2. Se le asigna de inmediato una etiqueta visible destacada: **BLOQUEADA**.
3. Se documenta formalmente en Jira: **Motivo del bloqueo** (causa técnica o dependencia), **Responsable** y **Acción requerida**.
4. El Scrum Master actúa prioritariamente en la Daily Scrum para levantar el impedimento con la mayor celeridad.

11.6. Dinámica por Sprints y Evolución del Tablero

11.6.1. Sprint 1 (HU-001 a HU-006): Núcleo de Capacitación y Acceso Público

El Sprint 1 inicia con la asignación simultánea de los tres desarrolladores activos:

TO DO	DESARROLLO [WIP: 3]	REVISIÓN [WIP: 2]	DONE
HU-004 – Previsualizar y publicar	HU-001 – Área interna (<i>Dev 1</i>)	(vacío)	(vacío)
HU-005 – Explorar públicas	HU-002 – Crear capacitación (<i>Dev 2</i>)		
HU-006 – Consultar pública	HU-003 – Slides interactivos (<i>Dev 3</i>)		

Tabla 6: Estado inicial del tablero Kanban al arranque del Sprint 1.

Conforme avanza la iteración, puede presentarse el siguiente estado de saturación en la columna de revisión:

TO DO	DESARROLLO [WIP: 3]	REVISIÓN [WIP: 2]	DONE
HU-006 – Consultar pública	HU-004 (<i>Dev 1</i>) HU-005 (<i>Dev 2</i>)	HU-002 HU-003	HU-001

Tabla 7: Escenario de saturación de Revisión (2/2) en el Sprint 1.

En este escenario, dado que Revisión = 2/2 (capacidad máxima alcanzada), **ningún desarrollador puede mover una nueva tarjeta a Revisión hasta que se libere al menos una**. La prioridad del equipo cambia inmediatamente: suspender el avance de nuevas tareas y colaborar para revisar, probar y fusionar HU-002 o HU-003 hacia desarrollo, desbloqueando el flujo de trabajo.

11.6.2. Sprint 2 (HU-007 a HU-012): Estructura Organizacional y Onboarding

El Sprint 2 arranca con HU-007 (Áreas y puestos), HU-008 (Rutas de onboarding) y HU-009 (Registro de colaboradores) en Desarrollo, mientras HU-010, HU-011 y HU-012 aguardan en To Do. Conforme se liberan las historias del primer bloque, las restantes ingresan ordenadamente garantizando que nunca se exceda el límite $WIP \leq 3$.

11.6.3. Sprint 3 (HU-013 a HU-015): Microtests Formativos y Supervisión

El Sprint 3 cuenta con 3 Historias de Usuario para 3 Desarrolladores ($WIP = 3/3$). Dado que HU-014 (Realizar microtest) depende funcionalmente de la estructura creada en HU-013 (Configurar microtest), el desarrollador responsable de HU-014 avanza en las pruebas (T1) e interfaz gráfica, sincronizando la integración de backend una vez consolidado el modelo de HU-013.

11.7. Política WIP Oficial del Equipo

- “1. La columna Desarrollo tendrá un límite máximo de 3 Historias de Usuario simultáneas, correspondiente a los tres integrantes del rol Development.
2. Cada desarrollador trabajará preferentemente sobre una única Historia de Usuario activa a la vez.
3. La columna Revisión tendrá un límite máximo de 2 Historias de Usuario simultáneas.
4. Cuando Revisión alcance su límite (2/2), el equipo priorizará de forma absoluta revisar, probar e integrar las historias existentes antes de iniciar o avanzar nuevas historias hacia dicha columna.
5. Las tarjetas impedidas permanecerán en su columna física identificadas con la etiqueta visible BLOQUEADA, indicando causa técnica, responsable y acción mitigadora.
6. Ninguna Historia de Usuario podrá pasar al estado DONE sin satisfacer al 100 % sus criterios de aceptación, pruebas automatizadas en verde y fusión validada en la rama desarrollo.”

11.8. Flujo Metodológico Integrado y Definition of Done

El ciclo global del proyecto se sintetiza en la siguiente secuencia de etapas:

BACKLOG → **SPRINT** → **TO DO** → **DESARROLLO [WIP 3]** → **REVISIÓN [WIP 2]** → **DONE**

y a nivel interno de cada Historia de Usuario en desarrollo:

T1: RED (Pruebas) → **T2: GREEN (Código)** → **T3: REFACTOR** → **T4: VALIDACIÓN / INTEGRACIÓN**

Una HU se declara formalmente **Done** si supera el 100 % de criterios de aceptación BDD, cuenta con pruebas automáticas en verde, no presenta errores de compilación en Next.js ni TypeScript, cuenta con persistencia validada en MongoDB Atlas vía Prisma ORM, y se encuentra integrada limpiamente en la rama **desarrollo**.

12. Incrementos Presentados del Proyecto

12.1. Incremento 1 (Sprint 1: Núcleo de Capacitación y Acceso Público)

- **HU-001 (Acceder al área interna):** Autenticación segura y redirección por rol (Dev 1: Carpio Hermoza, Alex).
- **HU-002 (Crear capacitación):** Gestión de datos generales y persistencia en MongoDB (Dev 2: Quispe Mamani, Domingo).
- **HU-003 (Slides interactivos):** Constructor de diapositivas con texto, imágenes y botones (Dev 3: Ticona Jancco, Ronaldo).
- **HU-004 (Publicar capacitación):** Previsualización interactiva y flujo de estados (Dev 1: Carpio Hermoza, Alex).
- **HU-005 (Explorar capacitaciones públicas):** Catálogo abierto para estudiantes y docentes (Dev 2: Quispe Mamani, Domingo).
- **HU-006 (Consultar capacitación pública):** Visualizador de slides interactivo sin autenticación (Dev 3: Ticona Jancco, Ronaldo).

Pruebas y Evidencias: Pruebas automatizadas sobre endpoints de Next.js, verificación de persistencia en MongoDB Atlas Cluster0 vía Prisma ORM y validación de los criterios CA-01 a CA-04 de cada historia bajo TDD.

13. Métricas y Seguimiento del Proceso

- **Velocidad del Sprint:** 6 HUs en Sprint 1, 6 HUs en Sprint 2 y 3 HUs en Sprint 3 (Total: 15 Historias de Usuario del MVP completadas).
- **Cycle Time:** Promedio de 3 a 4 días hábiles por historia desde **TO DO** hasta **DONE**.

- **Criterios Superados:** Tasa de éxito del 100 % en criterios de aceptación verificados.
- **Calidad de Código:** Cero regresiones en la rama **desarrollo** respaldadas por la suite TDD.

14. Acuerdos de Mejora para el Siguiete Incremento

1. Aprobar prototipos en Figma a más tardar el segundo día de cada sprint.
2. Fortalecer los tests de integración con Supertest para casos de sesión inválida.
3. Coordinar el esquema de base de datos relacional del Sprint 2 durante la sesión de Sprint Planning.

15. Política de Uso de IA y Registro de Uso

Fecha	Herramienta	Actividad / Propósito	Validación Técnica
30/09/2026	Gemini / Claude	Estructuración formal de la especificación técnica CGB Academy	Revisión contra el documento base oficial de 18 páginas.
30/09/2026	Copilot / ChatGPT	Formateo y tipografía del informe en LaTeX (<code>informe.tex</code>)	Compilación limpia con <code>pdflatex</code> y verificación de tablas.
30/09/2026	Antigravity Assistant	Automatización de scripts de verificación y ramas Git	Ejecución local y verificación de sincronización en GitHub.

Tabla 8: Bitácora de uso asistido de Inteligencia Artificial.

16. Conclusiones

1. **Eficacia del Enfoque Híbrido:** La integración de la cadencia de Scrum, la limitación del WIP de Kanban fundamentada en la Ley de Little y las prácticas de ingeniería de XP/TDD proporciona un flujo continuo y predecible para el equipo de desarrollo.
2. **Rigor en la Ingeniería de Requisitos y Calidad:** La formulación de historias de usuario bajo el estándar ISO/IEC/IEEE 29148 y el marco INVEST, combinada con la clasificación taxonómica de atributos de calidad ISO/IEC 25010:2023, asegura un producto robusto y verificable.
3. **Aseguramiento de Calidad y Deuda Técnica:** El ciclo TDD con 4 subtarefas técnicas obligatorias y el patrón Singleton en Prisma Client garantizan un código modular, mantenible y libre de sobrecargas de conexión en la infraestructura objetivo.

Referencias

- [1] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, Sequim, WA, 2010.

- [2] Kent Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston, 2000.
- [3] Mike Cohn. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, Boston, 2004.
- [4] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional, Boston, 2da edition, 2018.
- [5] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, 1994.
- [6] IEEE Computer Society. Guide to the software engineering body of knowledge (swebok guide), version 4.0. IEEE Computer Society Standard, 2024.
- [7] ISO/IEC. Iso/iec 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, 2023.
- [8] ISO/IEC/IEEE. Iso/iec/ieee 29148:2018 systems and software engineering — life cycle processes — requirements engineering, 2018.
- [9] ISTQB. Certified tester foundation level (ctfl) syllabus version 4.0, 2023.
- [10] Kanban University. The official kanban guide: The principles, practices, and rules of the kanban method. Kanban University Resources, 2025.
- [11] John D. C. Little. A proof for the queuing formula: $l = \lambda w$. *Operations Research*, 9(3):383–387, 1961.
- [12] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, Upper Saddle River, NJ, 2008.
- [13] Roger S. Pressman. *Ingeniería del Software: Un enfoque práctico*. McGraw-Hill, México D.F., 7ma edition, 2010.
- [14] Ken Schwaber and Jeff Sutherland. La guía de scrum: Las reglas del juego, 2020.
- [15] Ian Sommerville. *Ingeniería del Software*. Pearson Educación, Madrid, 9na edition, 2011.
- [16] Murugiah Souppaya, Karen Scarfone, and Donna Dodson. Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities. Technical Report NIST Special Publication 800-218, National Institute of Standards and Technology (NIST), 2022.